



Motors

By 9658, 6911, and the YASS team!

Two types of motors

Brushed

- DC power input
- **NO** integrated relative encoder
- **Not supported in YAMS**



Brushless

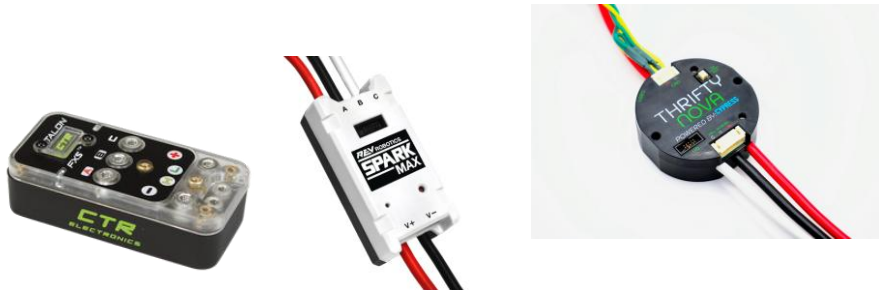
- 3 phase power input or integrated controller
- Integrated relative encoder
- Lighter and more energy efficient



Two types of motor controllers

External

- Outputs 3 phase or DC power
- Requires sensor cable
- Requires more space on the robot



Integrated

- Attached to the motor
- Integrated sensor handling
- Minimal extra space required
- More expensive to replace



Stalling Motors



Motor Stalling: Powering a motor, but instead of moving, the motor turns the unreleased potential energy into heat. Stall a motor for too long and you will release magic smoke. The bigger the motor, the longer it can be stalled.

You can measure how much your motor is stalling by checking the motors Stator Current

Small motors, such as Neo 550's should not be stalled!

Backlash

Backlash is the tiny gap between two parts that are supposed to mesh tightly, usually gears. When you change direction, the mechanism has to “take up” that gap before anything actually moves. That little delay feels like wiggle, slop, or looseness.

In robots, backlash shows up as:

The arm or elevator wobbling a bit when it stops

A delay between commanding motion and seeing motion

Difficulty tuning because the mechanism never behaves the same going forward and backward

A small amount is normal, but too much makes control messy and inconsistent.

Backlash will make your life harder if it hasn't already. When a mechanism has slop, you will have a much worse time tuning it and getting it where it needs to be will range from hard to impossible to get perfect. Here are some things to make sure the design and build team are keeping in mind:

- Pitch shifted pinions: for 20DP gears, the vendors sell a pinion that may say "8t (10t center distance)" which means that the tooth profile is altered to allow for smaller pinions than what would normally be possible. This can be very useful but try to never use these on an arm because they have a lot more backlash than normal pinions. It is usually okay for elevators because gravity is always pushing against the backlash. They also wear out faster and make more noise
- High ratio MAXPlanetary Stages (5:1, 6:1, 9:1) have a lot of backlash compared to the lower ratios. Try not to use those, or make sure you have more reduction after using one of those stages as it reduces the effective backlash. Planetaries will always have more backlash than a well-made custom gearbox

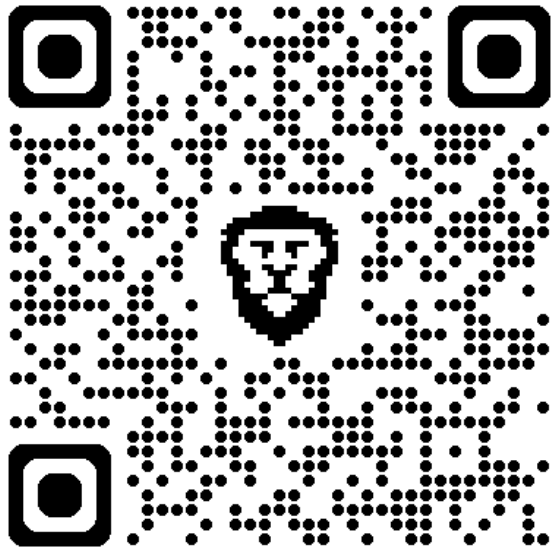
Legal Motors and Motor Controllers



- Not every motor controller you find is allowed to be used in FRC. Check the game manual and the official FIRST Community Blog for more information about legal and illegal motors and motor controllers.
- <https://community.firstinspires.org/2025-robot-rules-preview-for-2026>
- **YAMS ONLY SUPPORTS BRUSHLESS MOTORS AND MOTOR CONTROLLERS!**

Software

YAMS Documentation



<https://yagsl.gitbook.io/yams/documentation>



<https://yet-another-software-suite.github.io/YAMS/javadocs/>

SmartMotorController class

- YAMS provides the *SmartMotorController* class that standardizes motor control across vendor libraries like CTRE Phoenix 6, REVLib, or ThriftyLib into one class.
- This allows for you to configure and control all your motors the same way!
- Note: some configuration options may raise exceptions due to incomplete vendor parity and may require an extra step when you try to use open loop control functions like *setVoltage* or *setDutyCycle* when you use closed loop control as well.

```
/// Generic Smart Motor Controller with our options and vendor motor.  
private final SmartMotorController motor = new SparkWrapper(armMotor, dcMotor, motorConfig);
```

SmartMotorController class

- YAMS provides the *SmartMotorController* class that standardizes motor control across vendor libraries like CTRE Phoenix 6, REVLib, or ThriftyLib into one class.
- This allows for you to configure and control all your motors the same way!

Important: Some configuration options may raise exceptions due to incomplete vendor parity, and may require an extra step

```
/// Generic Smart Motor Controller with our options and vendor motor.  
private final SmartMotorController motor = new SparkWrapper(armMotor, dcMotor, motorConfig);
```

How to use the SmartMotorController

To make a SmartMotorController you need

1. Vendor Motor Object
2. Attached Motor (with count for accurate simulation)
3. *SmartMotorControllerConfig*

```
motor = new SparkWrapper(new SparkMax( deviceld: 1, MotorType.kBrushless),
    DCMotor.getNEO( numMotors: 1),
    smartMotorControllerConfig);

DCMotor minion = new DCMotor( nominalVoltageVolts: 12,
    stallTorqueNewtonMeters: 3.1,
    stallCurrentAmps: 200.46,
    freeCurrentAmps: 1.43,
    RPM.of( magnitude: 7200).in(RadiansPerSecond),
    1); // numMotors

motor = new TalonFXWrapper(new TalonFXS( deviceld: 1),
    minion,
    smartMotorControllerConfig);

motor = new NovaWrapper(new ThriftyNova( canID: 1),
    DCMotor.getNEO( numMotors: 1),
    smartMotorControllerConfig);

motor = new TalonFXWrapper(new TalonFX( deviceld: 1),
    DCMotor.getKrakenX60( numMotors: 1),
    smartMotorControllerConfig);
```

Using SmartMotorController pt2

YAMS Supports:

- SparkMax
- SparkFlex
- ThriftyNova
- TalonFX
- TalonFXS

If you need to create the DCMotor use
<https://reca.lc/motors>

SmartMotorController

There are 4 ways to control motors

1. Position
2. Velocity
3. DutyCycle
4. Voltage

Position and Velocity Overloads exist!

- a. Angle based for Arms, Turrets, Wrists, etc
- b. Distance based for Elevators, Linear Actuators, etc

```
motor.setPosition(Degrees.of(magnitude: 30));  
motor.setPosition(Meters.of(magnitude: 3));  
  
motor.setVelocity(RPM.of(magnitude: 10000));  
motor.setVelocity(MetersPerSecond.of(magnitude: 1));  
  
motor.setDutyCycle(0.5);  
  
motor.setVoltage(Volts.of(magnitude: 7));
```

SmartMotorControllerConfig

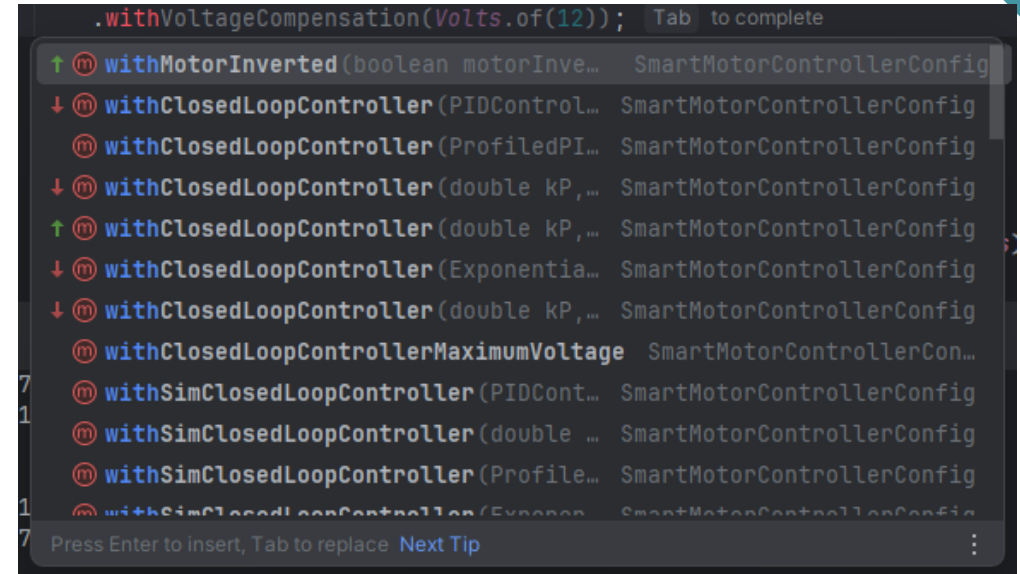
SmartMotorControllerConfig is how to configure each *SmartMotorController*

NOT ALL FEATURES ARE SUPPORTED ON EVERY *SmartMotorController*

The *SmartMotorControllerConfig* can be broken up into 5 categories

1. Motor
2. Closed Loop
3. Encoder
4. External Encoder
5. Telemetry

Every configuration option is prefexed with ".with" so if you would like to discover any option just use intellisense!



```
.withVoltageCompensation(Volts.of(12)); Tab to complete
↑ (m) withMotorInverted(boolean motorInve... SmartMotorControllerConfig
↓ (m) withClosedLoopController(PIDControl... SmartMotorControllerConfig
(m) withClosedLoopController(ProfiledPI... SmartMotorControllerConfig
↓ (m) withClosedLoopController(double kP,... SmartMotorControllerConfig
↑ (m) withClosedLoopController(double kP,... SmartMotorControllerConfig
↓ (m) withClosedLoopController(Exponentia... SmartMotorControllerConfig
↓ (m) withClosedLoopController(double kP,... SmartMotorControllerConfig
(m) withClosedLoopControllerMaximumVoltage SmartMotorControllerCon...
7 (m) withSimClosedLoopController(PIDCont... SmartMotorControllerConfig
1 (m) withSimClosedLoopController(double ... SmartMotorControllerConfig
(m) withSimClosedLoopController(Profile... SmartMotorControllerConfig
1 (m) withSimClosedLoopController(Exponen... SmartMotorControllerConfig
7 Press Enter to insert, Tab to replace Next Tip
```

SmartMotorControllerConfig - Motor

- There are relatively few motor specific configuration options that don't need much explanation.
 - *withControlMode*
 - *withFollowers*
 - *withIdleMode*
 - *withMomentOfInertia*
 - *withMotorInverted*
 - *withOpenLoopRampRate*
 - *withClosedLoopRampRate*
 - *withStatorCurrentLimit*
 - *withSupplyCurrentLimit*
 - *withVoltageCompensation*
 - *withTemperatureCutoff*

How do I add follower motors?

Follower motors are only able to be set IF the motor is the same vendor. This is because only alike vendor motor controllers can “follow” the main motor controller.

When defining a follower motor you can inverse it very easily as well!

You can define as many followers as you want in the *withFollowers* function parameters too!

```
private final SmartMotorControllerConfig smartMotorControllerConfig = new SmartMotorControllerConfig( subsystem: this)
    .withFollowers(Pair.of(new SparkFlex( deviceId: 1, MotorType.kBrushless), b: false))
    .withControlMode(ControlMode.CLOSED_LOOP)
    .withClosedLoopController( kP: 1, kI: 0, kD: 0, RPM.of( magnitude: 10000), RPM.per(Second).of( magnitude: 60))
    .withFeedforward(new SimpleMotorFeedforward( ks: 0, kv: 0, ka: 0))
```

What is my motor's moment of inertia?

The moment of inertia is used to more accurately simulate your motors behavior under load.

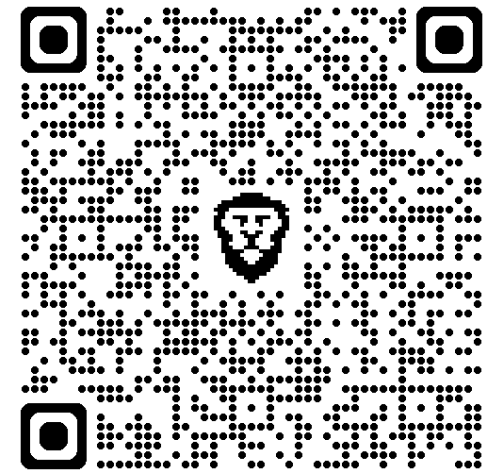
It is easy to define as the MOI is composed of the distance and weight of the mechanism.

```
SmartMotorControllerConfig smartMotorControllerConfig = new SmartMotorControllerConfig(subsystem: this)
    .withMomentOfInertia(Inches.of(magnitude: 4), Pounds.of(magnitude: 0.3))
    .withMomentOfInertia(MOI: 1)
```

Stator Current vs Supply Current

- Stator Current: How much current the stator, the spinning part, is seeing. This include the current used by the motor brake, usually represented by a negative current.
- Supply Current: How much power the controller is pulling from the 12V wires

```
SmartMotorControllerConfig smartMotorControllerConfig = new SmartMotorControllerConfig( subsystem: this)  
.withStatorCurrentLimit(Amps.of( magnitude: 100))  
.withSupplyCurrentLimit(Amps.of( magnitude: 40))
```



Voltage Compensation

Voltage Sag

When we operate mechanisms on our robot, we draw current from its battery. This causes the available “bus voltage” that all the robot mechanisms operate off of to drop. This means that the performance of our mechanisms will vary depending on the loading and action of the robot - this is not ideal.

To fix this, most voltage controllers offer a “voltage compensation” setting for their internal control loops that keep the output voltage of the control loops constant despite changes in the bus voltage.

```
SmartMotorControllerConfig smartMotorControllerConfig = new SmartMotorControllerConfig( subsystem: this)  
.withVoltageCompensation(Volts.of( magnitude: 12))
```

Voltage Compensation

Keep in mind that voltage compensation cannot increase the voltage applied to the motor beyond what is available on the bus - if your actuator is saturating (described below), you'll have to account for that separately.

Actuator Saturation

A controller calculates its output based on the error between the [setpoint](#) and the current [state](#). [Plant](#) in the real world don't have unlimited control authority available for the controller to apply - that is to say, real mechanisms have some maximum achievable torque/acceleration and velocity.

If our control gains are too aggressive, our control algorithm might try to move the mechanism faster than it is capable of actually going. In this case, the mechanism will "saturate", and behave as if the control gains were smaller than they are. This might adversely affect control response (i.e., result in errors and instability).

If you are encountering problems with actuator saturation, consider modifying your mechanism gearing or powering it with a bigger motor.

Temperature Cutoff

- The only motor controller which supports this currently is the SparkMax and SparkFlex.
- This allows for the motor to set the voltage to 0 in the closed loop if a temperature is exceeded potentially saving your motor.
- Unfortunately, most vendors do not support this yet so please read your error messages carefully and simulate your program first!

```
private final SmartMotorControllerConfig smartMotorControllerConfig = new SmartMotorControllerConfig(subsystem: this)
    .withTemperatureCutoff(Fahrenheit.of((200)))
```

Ramp Rates and proper control

In general **Ramp Rates** are **NOT RECOMMENDED** they are often band-aids to structural problems.

Ramp Rates slow down your control input which is very detrimental to the speed of your motor and by proxy your mechanism.

Instead of using **Ramp Rates** you should use **Stator Current Limits**, and **Motion Profiles** like Trapezoidal Profiles or Exponential Profiles.

There are 2 types of **Ramp Rates**

- Open Loop
 - ☐ Applied when you control your motor via voltage or dutycycle, skews the input to be spread out over X seconds.
- Closed Loop
 - ☐ Applied when you control your motor via setpoints with closed loops, interpolates your setpoint to be spread out over X seconds.

Idle Mode

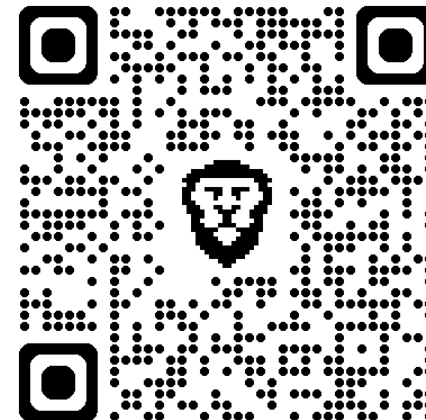
Idle Modes define what the motor does when it is at 0 power.

There are two types of idle modes:

- Brake Mode: When the motor output is set to not move, it would hold its place. It does this by shorting the motor wires together. It is not mechanically locked so the motor can be overcome with high torque.

- Coast Mode: Motor Breaking is not engaged at 0 power.

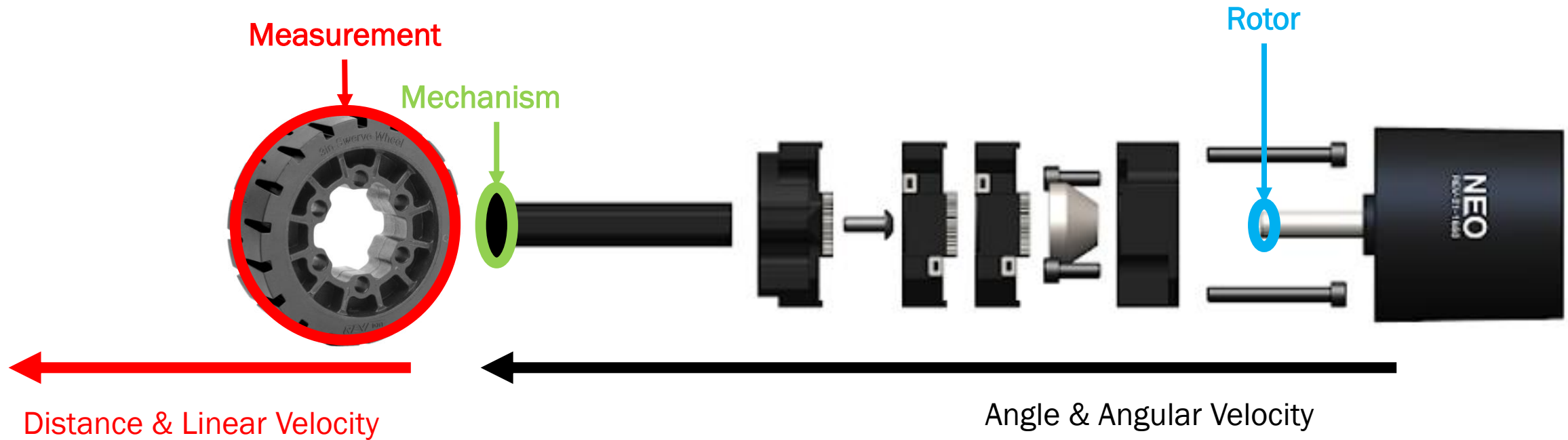
<https://bovlb.github.io/frc-tips/coast-mode/>



SmartMotorControllerConfig – Encoder

- Magnetic: Uses a sensor to measure the rotation of a dipole magnet. Must be placed at the ends of the shaft.
- Through bore: Directly attaches to the rotation shaft and can be anywhere along the center.
- Relative: Sensor is unable to get position of power on and starts at zero, internal motor encoder
- Absolute: Can get the sensor's rotation on power on, mag encoder.

What is my Gearing?



What is my Gearing?

Your gearing is the ratio of the input to the output, usually expressed in rotations, including sprockets and gearboxes!

We try to make this super easy to define so if anything changes during the season it doesn't take forever to calculate your new gear ratio.

The reductionRatio is how many rotations it takes for your motor to rotate your mechanism.

- The Gearing is applied to the motor controller and used to get the “Mechanism” position and velocity.

```
motor = new SparkWrapper(new SparkMax( deviceId: 1, MotorType.kBrushless),
                          DCMotor.getNEO( numMotors: 1),
                          smartMotorControllerConfig);
motor.getMechanismVelocity();
motor.getMechanismPosition();
```

```
SmartMotorControllerConfig smartMotorControllerConfig = new SmartMotorControllerConfig( subsystem: this)
.withGearing( reductionRatio: 21.48572)
.withGearing(new MechanismGearing(GearBox.fromReductionStages(3, 4, 5)))
.withGearing(new MechanismGearing(GearBox.fromReductionStages(3), Sprocket.fromStages("12:32", "124:210")))
.withGearing(new MechanismGearing(GearBox.fromStages("9:1", "3:1")))
```

What is my Measurement?

Measurement's are treated as Mechanism readings (post gearbox/sprocket) translated into distance via the supplied or calculated circumference.

If you try to use the measurement without setting the *SmartMotorControllerConfig* like below it will crash because it can't.

```
motor = new SparkWrapper(new SparkMax( deviceId: 1, MotorType.kBrushless),  
                           DCMotor.getNEO( numMotors: 1),  
                           smartMotorControllerConfig);  
motor.getMeasurementPosition().in(Meters);  
motor.getMeasurementVelocity().in(MetersPerSecond);
```

```
SmartMotorControllerConfig smartMotorControllerConfig = new SmartMotorControllerConfig( subsystem: this)  
    .withMechanismCircumference(Inches.of( magnitude: 4).times(Math.PI))  
    .withWheelDiameter(Inches.of( magnitude: 4))  
    .withWheelRadius(Inches.of( magnitude: 2))
```

Encoder Wrapping/Continuous Wrapping

- Absolute encoders will always wrap around a point on the sensor.
- *withContinuousWrapping* will tell the PID controller that the sensor wraps around from the max to the min.
 - Valid options are [-0.5, 0.5), [-1, 0), [0, 1) in Rotations.
 - Note that it will never reach the max point!

```
SmartMotorControllerConfig smartMotorControllerConfig = new SmartMotorControllerConfig( subsystem: this)
.withContinuousWrapping(Rotations.of( magnitude: -0.5), Rotations.of( magnitude: 0.5))
.withContinuousWrapping(Rotations.of( magnitude: 0), Rotations.of( magnitude: 1))
.withContinuousWrapping(Rotations.of( magnitude: -1), Rotations.of( magnitude: 0))
```

Soft Limits vs Hard Limits

- SoftLimit: Uses encoders to measure how far it has moved and digitally does not allow motor to rotate past its sensor value. Requires encoders to be set up correctly and undamaged.
- HardLimit: Uses a physical button to stop motor if button is pressed. Usually placed at the ends of a mechanism. Mechanisms can have inertia and move past the Hard limit after motors is commanded to stop.
 - Simulation prevents movement past hard limits in YAMS. Only supported on mechanisms.
- HardStop: Physically prevents the mechanism to continue moving by adding bumpers or metal at the end of travels. Useful if something goes wrong and motors are being commanded to move past the limits.

```
SmartMotorControllerConfig smartMotorControllerConfig = new SmartMotorControllerConfig(subsystem: this)
.withSoftLimit(Degrees.of(magnitude: -30), Degrees.of(magnitude: 100))
```

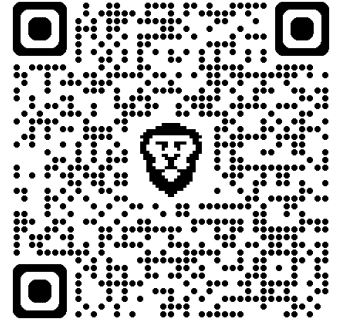
SmartMotorControllerConfig – Closed Loop

Closed Loop Controllers are known as Feedback Controllers and are excellently documented at WPILib.

For YAMS you can define them *.withClosedLoopController* and *.withSimClosedLoopController* for sim only closed loop values which may differ from real life.

```
SmartMotorControllerConfig smartMotorControllerConfig = new SmartMotorControllerConfig( subsystem: this)
.withClosedLoopController( kP: 0, kI: 0, kD: 0)
.withClosedLoopController( kP: 0, kI: 0, kD: 0, RPM.of( magnitude: 10000), RPM.per(Second).of( magnitude: 60))
.withClosedLoopController(new ExponentialProfilePIDController( kP: 0,
                                                                kI: 0,
                                                                kD: 0,
                                                                ExponentialProfilePIDController.createConstraints(
                                                                    Volts.of( magnitude: 12),
                                                                    RPM.of( magnitude: 1000),
                                                                    RPM.per(Second).of( magnitude: 60))))
.withWheelRadius(Inches.of( magnitude: 2))
.withClosedLoopController( kP: 0, kI: 0, kD: 0, MetersPerSecond.of( magnitude: 2), MetersPerSecondPerSecond.of( magnitude: 1))
```

Feedback (PID)



How do the PID parameters affect system dynamics?

The effects of increasing each of the controller parameters K_P , K_I and K_D can be summarized as

Response	Rise Time	Overshoot	Settling Time	S-S Error
K_P	Decrease	Increase	NT	Decrease
K_I	Decrease	Increase	Increase	Eliminate
K_D	NT	Decrease	Decrease	NT

NT: No definite trend. Minor change.

You may want to take notes of this table. It will be useful in the later part of the lesson.

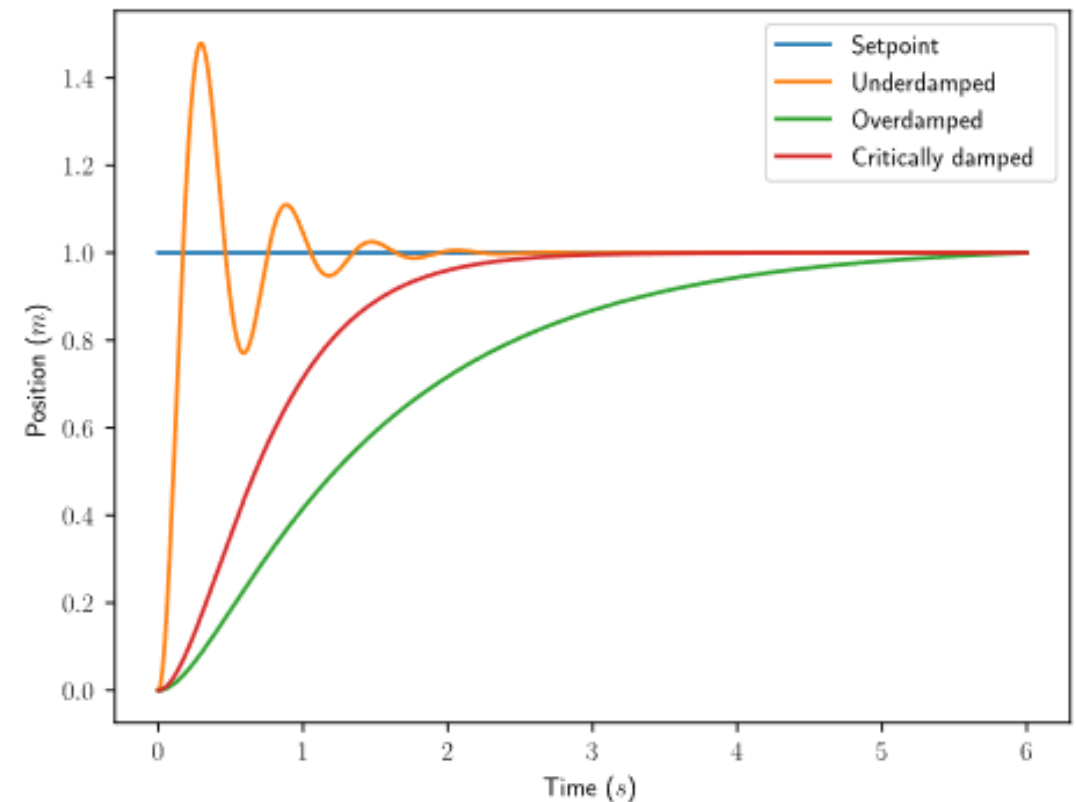
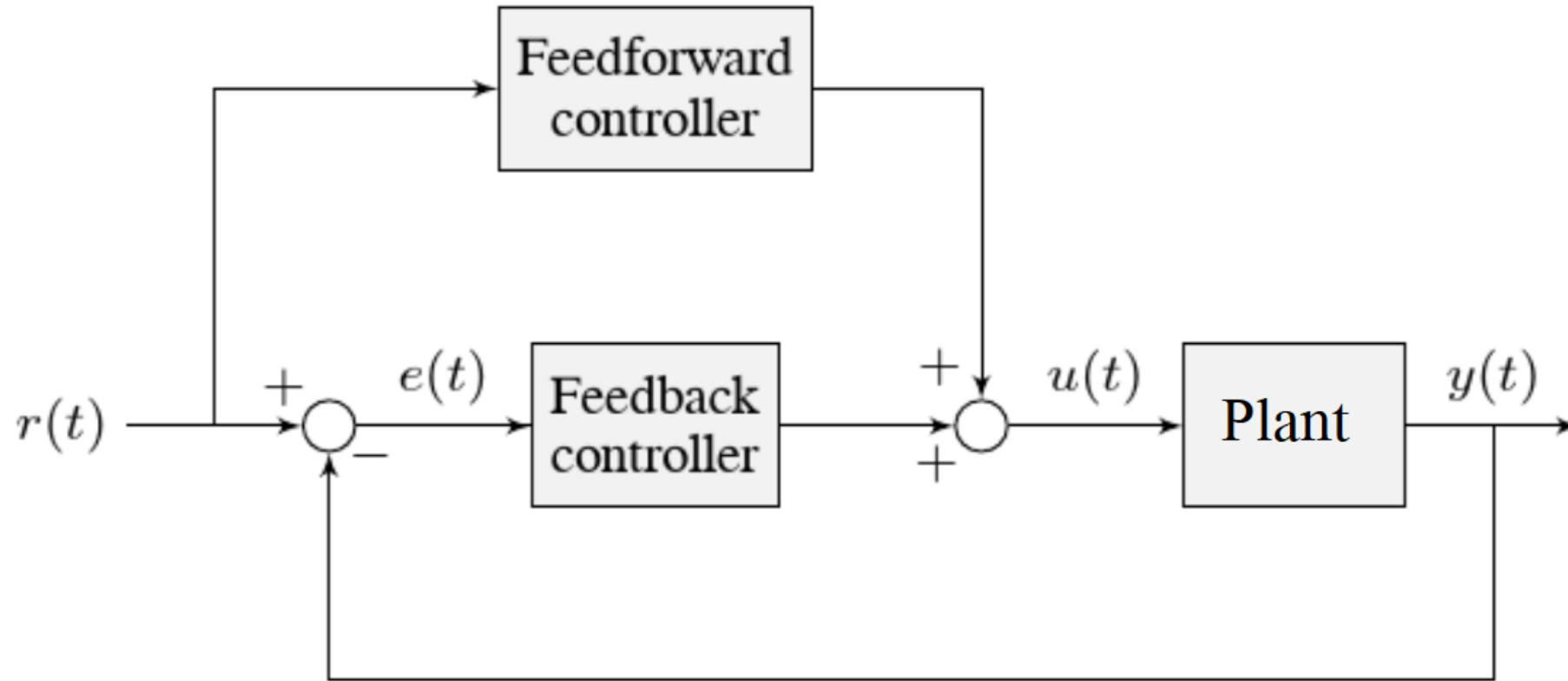
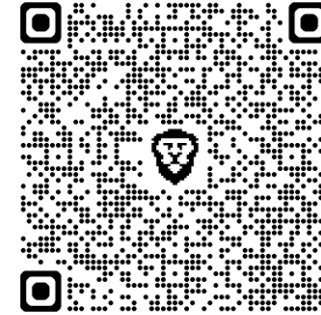


Figure 2.8: PID controller response types

Feedforward



Feedforward



$$V = K_s \cdot \text{sgn}(\dot{d}) + K_v \cdot \dot{d} + K_a \cdot \ddot{d}$$

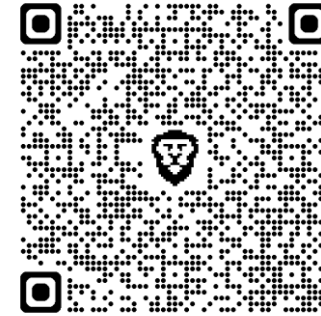
Where V is the applied voltage, d is the displacement (position) of the motor, $\dot{d}$ is its velocity, and $\ddot{d}$ is its acceleration (the “overdot” notation traditionally denotes the derivative with respect to time).

We can interpret the coefficients in the above equation as follows:

K_s is the voltage needed to overcome the motor’s static friction, or in other words to just barely get it moving; it turns out that this static friction (because it’s, well, static) has the same effect regardless of velocity or acceleration. That is, no matter what speed you’re going or how fast you’re accelerating, some constant portion of the voltage you’ve applied to your motor (depending on the specific mechanism assembly) will be going towards overcoming the static friction in your gears, bearings, etc; this value is your K_s .

Note the presence of the signum function because friction force always opposes the direction-of-motion.

Feedforward



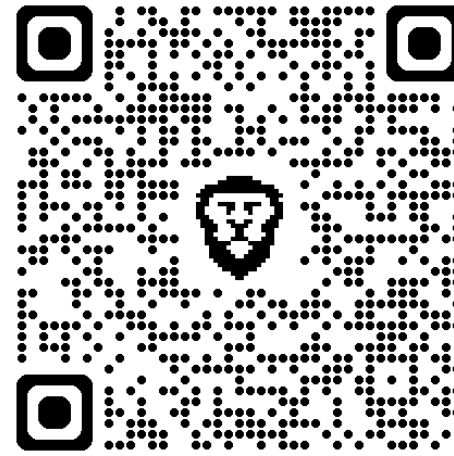
$$V = K_s \cdot \text{sgn}(\dot{d}) + K_v \cdot \dot{d} + K_a \cdot \ddot{d}$$

kV describes how much voltage is needed to hold (or “cruise”) at a given constant velocity while overcoming the counter-electromotive force and any additional friction that increases with speed (including viscous drag and some churning losses). The relationship between speed and voltage (at constant acceleration) is almost entirely linear (for FRC-legal components) because of how permanent-magnet DC motors work.

kA describes the voltage needed to induce a given acceleration in the motor shaft. As with kV, the relationship between voltage and acceleration (at constant velocity) is almost perfectly linear for FRC components.

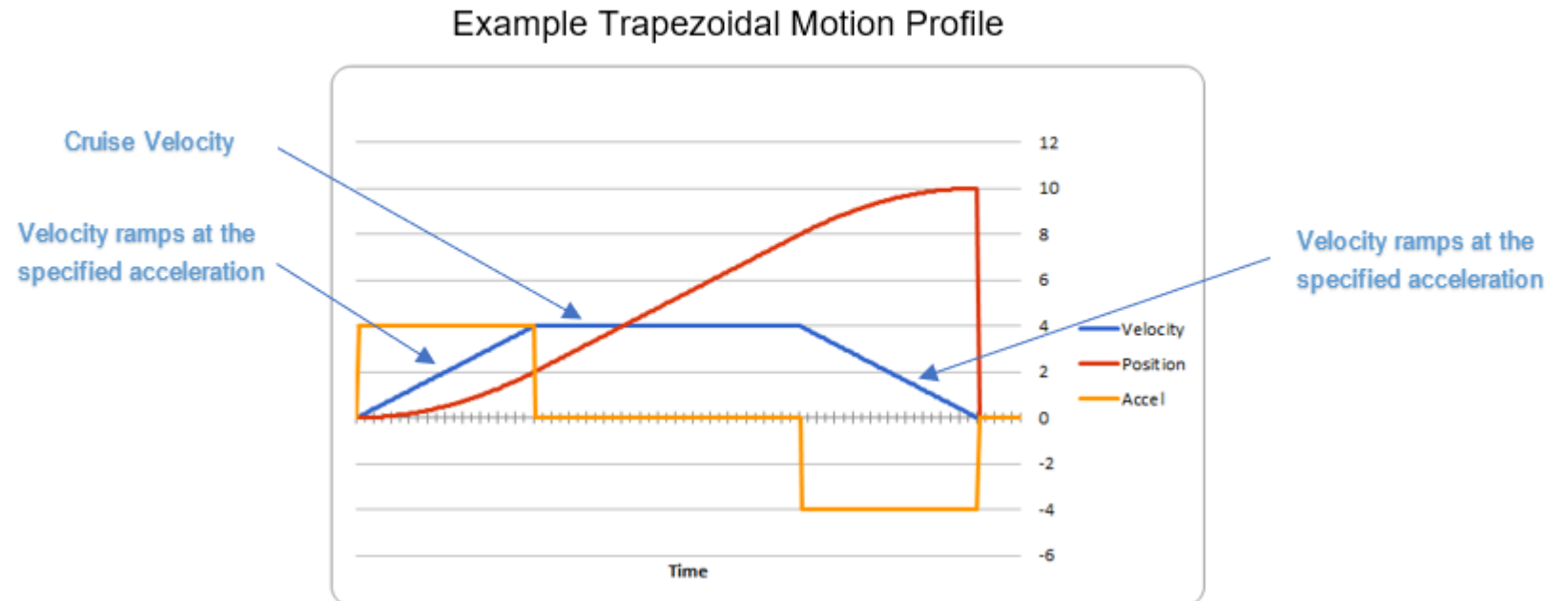
kG describes the voltage needed to resist the forces of gravity. Very useful for elevators.

Trapezoidal Profile

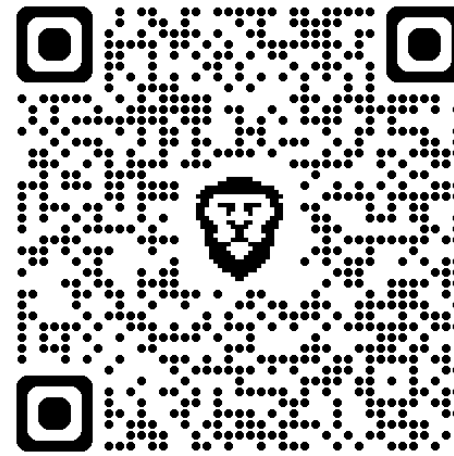


Trapezoidal Profiles limit the acceleration and velocity of a Feedback and Feedforward control scheme to smooth control.

The constraints on a Trapezoidal Profile are the max velocity and max acceleration.



Trapezoidal Profile

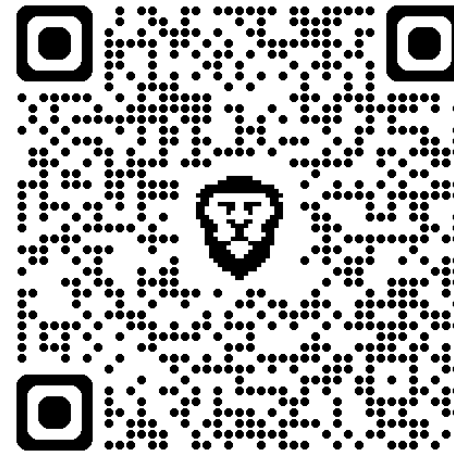


The benefits of this control mode over “simple” PID position closed-looping are:

- Control of the mechanism throughout the entire motion (as opposed to racing to the end target position);
- Control of the mechanism’s inertia to ensure smooth transitions between setpoints; Improved repeatability despite changes in battery load
- Improved repeatability despite changes in motor load

```
SmartMotorControllerConfig smartMotorControllerConfig = new SmartMotorControllerConfig( subsystem: this)
.withClosedLoopController( kP: 0, kI: 0, kD: 0, RPM.of( magnitude: 10000), RPM.per(Second).of( magnitude: 60))
.withWheelRadius(Inches.of( magnitude: 2))
.withClosedLoopController( kP: 0, kI: 0, kD: 0, MetersPerSecond.of( magnitude: 2), MetersPerSecondPerSecond.of( magnitude: 1))
```

Exponential Profile

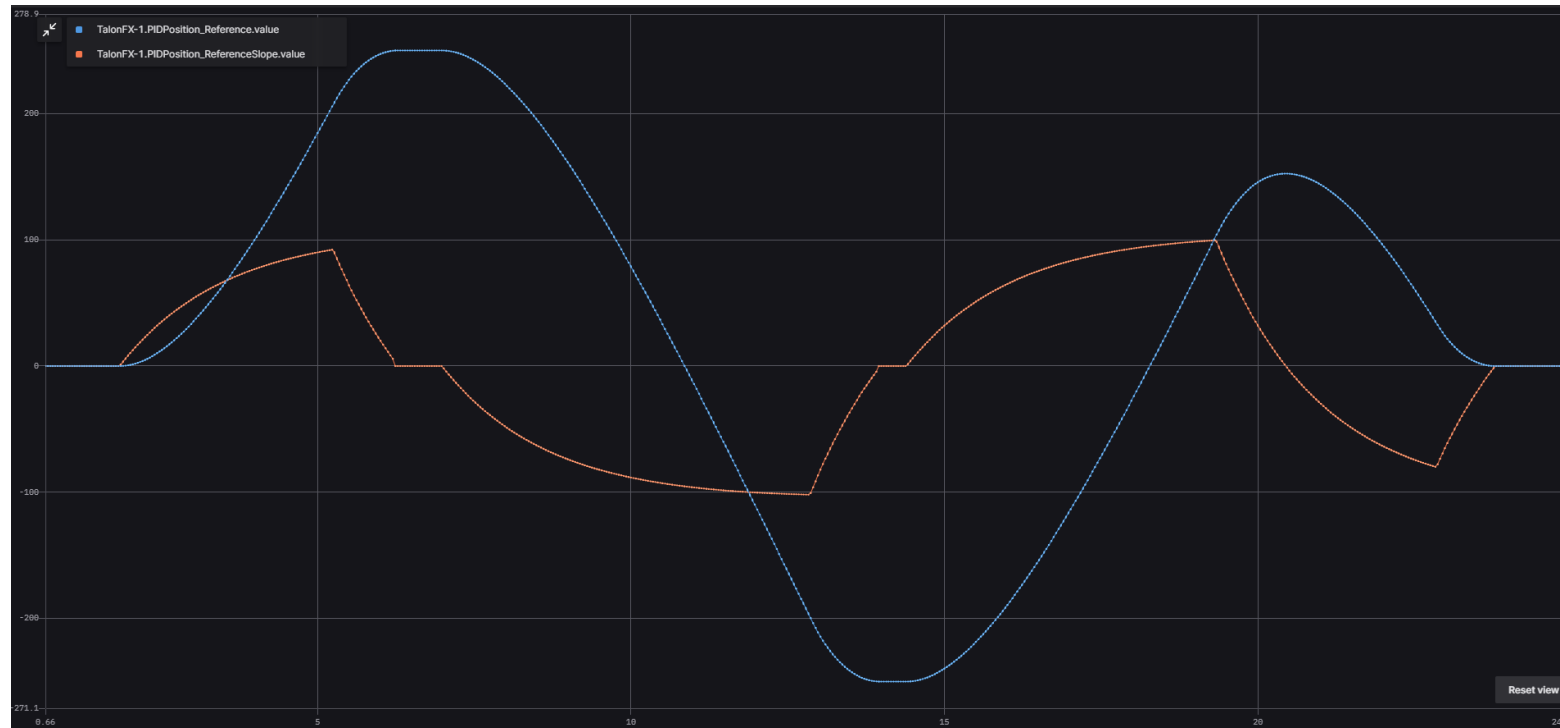


Exponential Profiles allow for smoother control over rougher mechanisms that experience some friction that the feedforward and feedback loops alone can't easily overcome.

The constraints are kV and kA .

kV represents the voltage required to maintain a given velocity.
(Voltage/AngularVelocity)

kA represents the voltage required to apply a given acceleration.
(Voltage/AngularAcceleration)





- Dividing the supply voltage by kV results in the maximum velocity of the profile.
- When supply voltage is fixed, a **higher profile kV** results in a **lower profile velocity**
- It is safer to start from a higher kV than what is ideal.

[illegible]



The parameters for *ExponentialProfilePIDController.createElevatorConstraints* are:

Max Volts

DCMotor

Carriage Weight

Elevator Height

Gearing

[illegible]



Max Volts
DCMotor
Flywheel Weight
Wheel Radius
Gearing

[illegible]



Max Angular Velocity (kV)

Max Angular Acceleration (kA)

[illegible]

Tuning



Tuning

Prerequisite for CTRE users: if you do not want to splitscreen Advantagescope with Phoenix when tuning, you can publish the Phoenix 6 signals that you need by adding the following (the graphs will be lower resolution than what you can see in Phoenix but should be able to get you 99% of the way there):

- 1) After creating the motor(s) in the subsystem:

```
private final StatusSignal<edu.wpi.first.units.measure.Angle> elevatorPositionSignal =  
    | elevatorMotor.getPosition();  
private final StatusSignal<Double> elevatorReferenceSignal =  
    | elevatorMotor.getClosedLoopReference();
```

- 2) In the public ElevatorSubsystem()

```
BaseStatusSignal.setUpdateFrequencyForAll(  
    | frequencyHz:50, elevatorPositionSignal, elevatorReferenceSignal);
```

- 3) In periodic() (requires the Advantagekit vendor dependency)

```
BaseStatusSignal.refreshAll(elevatorPositionSignal, elevatorReferenceSignal);  
Logger.recordOutput(  
    | key:"Elevator/FX/PositionRot", elevatorPositionSignal.getValue().in(Units.Rotations));  
Logger.recordOutput(key:"Elevator/FX/ReferenceRot", elevatorReferenceSignal.getValueAsDouble());
```

Tuning Order

1. Limit max velocity and acceleration during tuning to prevent damage, set k_G , k_V , k_A , k_P , k_I , k_D to Zero
2. Find the k_G that holds the mechanism in the air
3. Find the k_V that allows the velocity portions of the position and reference graphs to have the same slope
4. Find the k_A that allows the acceleration portions of the position and reference graphs to have the same slope (you mostly only care about the acceleration part here not the deceleration (yet))
5. Increase k_P until the mechanism overshoots and then back it off until the overshoot stops
6. Optionally you can add in some k_D to stop the overshoot, and you can also try to further crank the k_P and keep adding D until the mechanism starts to jitter around the setpoint

tip: for the gains other than k_G you will want the mechanism to start moving from wherever the reference setpoint is, to compare the graphs more easily. To do this, set the starting position, let the mechanism get close to it, then set the k_P to something high to get it to move the rest of the way then set the k_P back to whatever it was (0 until you are at the k_P tuning step)

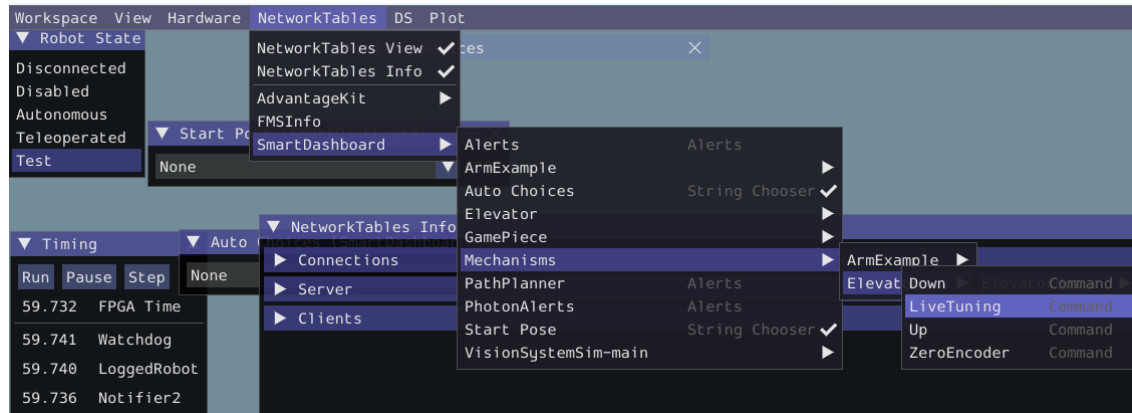
You do not need k_S in SIM, and for the most part you do not need to worry about k_S on a well-designed mechanism, especially elevators. It is more commonly going to be useful for arms and pivots

k_I is an error accumulator, Make sure to clear to accumulator before setting it to a value from zero.

Live Tuning

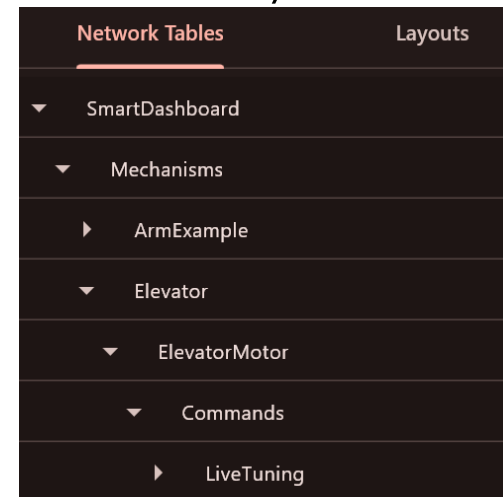
For both SIM and REAL, you need to be in test mode to use YAMS live tuning

SIM:



REAL: (you can also do it this way in SIM)

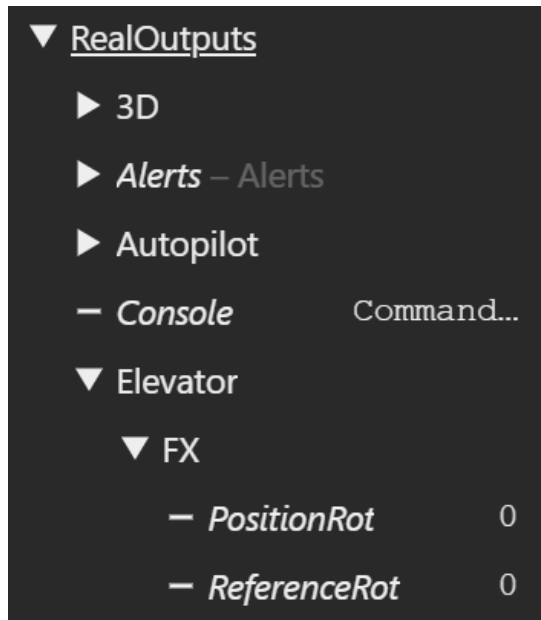
- 1) Add the Elastic Widget for Live Tuning to turn it on/off



You can also add widgets to tune in Elastic, but Advantagescope is recommended (next slide)

Tuning in Advatagescope

Open a line graph, and drag in the signals you need for a given mechanism to the left axis (the below example is if you are using CTRE motors)

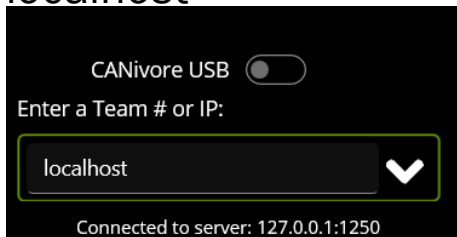


To be able to change the gains in AdvantagScope, click the button with 3 lines. It will turn purple when you can type things in



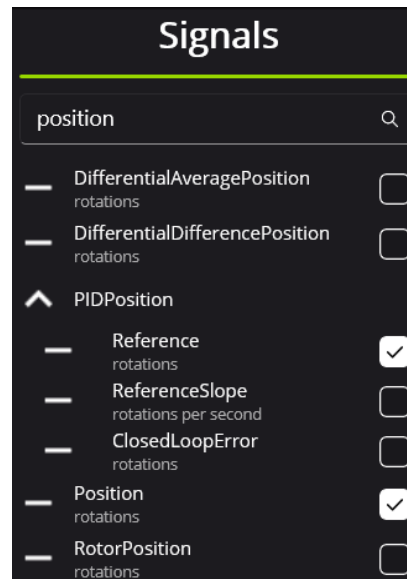
Tuning in Phoenix

In the menu, make sure you are connected to the correct data source. In SIM that is localhost

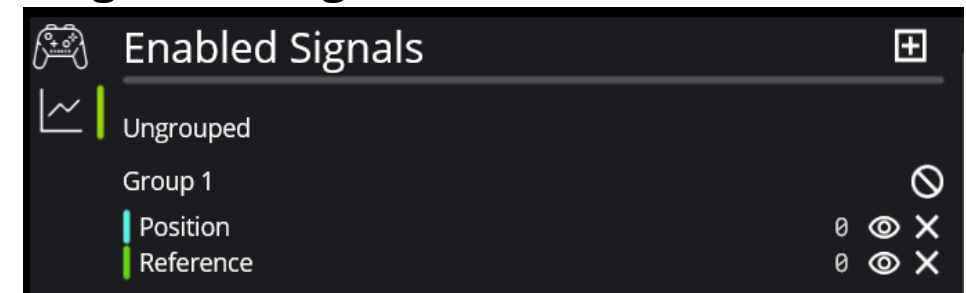


Click on the motor whose mechanism you want to tune, and open the plot

Open the Signals menu by clicking on the 3 dots in the top right corner, search "position" and add the Position and Reference signals:



Create a group by clicking the plus sign and drag the two signals into it so the axes match



MAKE SURE YOU HIT PLAY

Setup

You want to tune mechanisms (at least most of the tuning) with them going slow and set up soft limits if possible

Elevators: set velocity and acceleration to 0.3 (m/s and m/s²)

Arms: set acceleration and velocity to 90 (deg/s and deg/s²)

MAKE SURE TO SET ALL OF YOUR GAINS TO ZERO.
When you deploy for tuning, make them zero in code.
Open the tuning menu in AdvantageScope and expand the dropdowns for the mechanism you want to tune

The screenshot shows the 'Tuning' menu in AdvantageScope. It is expanded to show the 'ArmExample' section, which contains a 'Turret' dropdown. The 'Turret' dropdown is expanded to show the 'TurretMotor' section. The 'TurretMotor' section is expanded to show the 'closedloop' section. The 'closedloop' section is expanded to show the 'feedback' section. The 'feedback' section contains three gain settings: kD , kI , and kP , all set to 0. The 'closedloop' section is also expanded to show the 'feedforward' section. The 'feedforward' section contains four gain settings: kA , kG , kS , and kV , all set to 0. The 'closedloop' section is also expanded to show the 'motionprofile' section. The 'motionprofile' section contains two settings: $maxAcceleration$ and $maxVelocity$, both set to 90. The 'closedloop' section is also expanded to show the 'setpoint' section. The 'setpoint' section contains two settings: $position$ and $velocity$, both set to 0.

- ▼ Tuning
 - ▶ ArmExample
 - ▼ Turret
 - ▼ TurretMotor
 - ▼ closedloop
 - ▼ feedback
 - kD 0
 - kI 0
 - kP 0
 - ▼ feedforward
 - kA 0
 - kG 0
 - kS 0
 - kV 0
 - ▼ motionprofile
 - $maxAcceleration$ 90
 - $maxVelocity$ 90
 - ▼ setpoint
 - $position$ 0
 - $velocity$ 0

Tuning kG

Basically, you want to increase kG until the mechanism starts moving, then back off until it stops moving.

- Command a setpoint and then start incrementally increasing kG. With all your gains at zero, kG should be able to hold the mechanism in the air

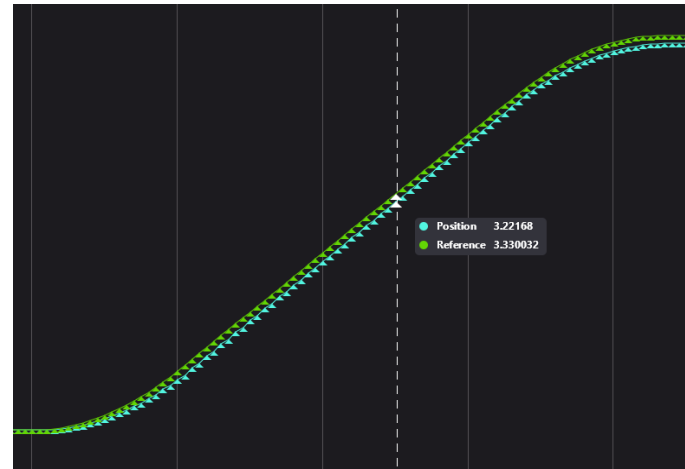
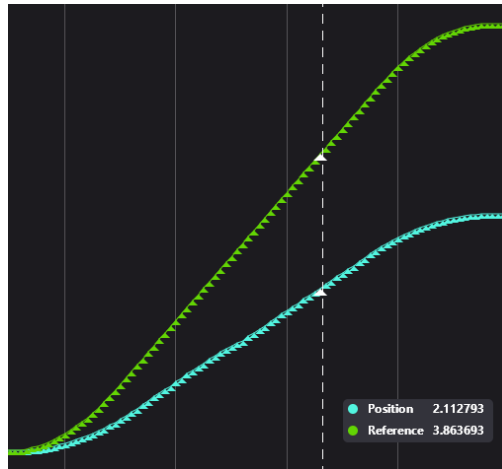
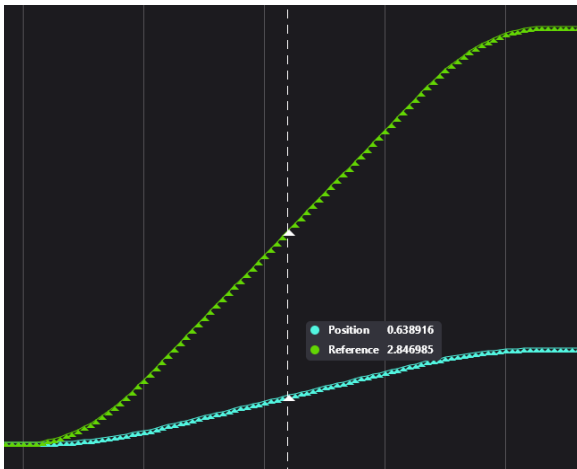
You should expect to tune kG to 2 or 3 decimal places

Tuning kV

Start with a low kV (0.1) and increase in increasingly large steps

Below are 2 graphs with kV that is too low and one that is correct. With the kV too low, the slope of the position is shallower than the slope of the reference and it undershoots by a lot

When the kV is correct, the slopes match and the mechanism nearly reaches the setpoint

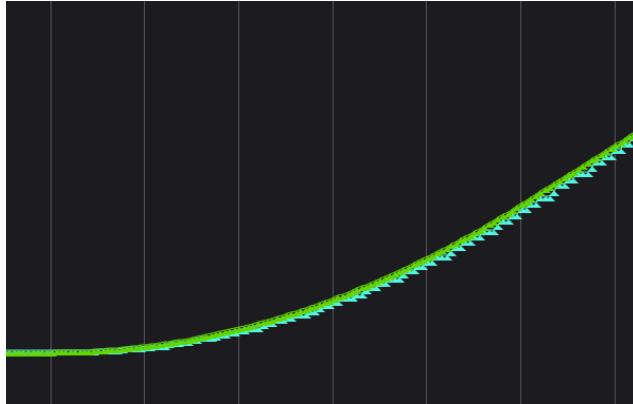
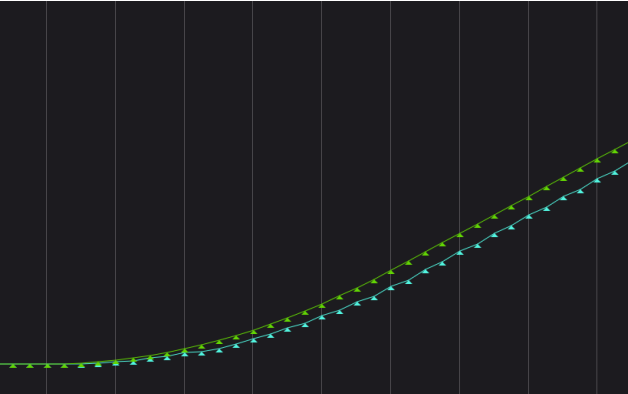


Tuning kA

The main objective with kA is to get the graphs to overlap by increasing the acceleration of the mechanism.

kA will generally be very low. Start with 0.001

This part of the graph is what you want to use kA to match. Here, kA is zero



Tuning k_P and k_D



Now, the graphs should match until the mechanism approaches the setpoint, at which point it will stop prematurely. To solve this increase k_P until the mechanism begins to overshoot and then back off k_P until the overshoot stops.

After this, begin to increase the acceleration and velocity. Once the mechanism is moving at full speed, you can add some k_D , which will let you further increase k_P

Adding k_D when the mechanism is moving slow is not going to be very informative as the system dynamics are pretty tame at this point. k_D helps to reduce the oscillating around the setpoint that you can see with high-speed mechanisms.

Why not just lower k_P instead of adding k_D ? The lower k_P is, the slower the control loop will correct small errors. When k_P is low, overshoot is prevented by now allowing the mechanism to approach the setpoint more gently. By increasing k_P and adding k_D , the mechanism will move much faster and then "slam the breaks" to stop more aggressively

On lightweight, slow mechanisms, you can absolutely get away with just P, but as speed and inertia increase, D becomes your friend

Characterizing Your Mechanisms With SysId

WPILib's SysID (system identification) routines can be run to determine your feedback and feedforward gains by running a series of tests while collecting motor data. You can learn more about that [here](#). 4 tests get ran: quasistatic forward, quasistatic reverse, dynamic forward, and dynamic reverse.

YAMS makes it extremely easy to run these tests by pressing a single button, but it relies on your limits being set correctly to know when to switch from forward to reverse and vice versa. If your limits are not correctly set, the command may run the mechanism into its hard limits, assuming those exist to at least prevent destruction. When running SysId it is recommended to alter your hard limit configs to be slightly within the true hard limits so the mechanism does not hit any end stops during the test, which will negatively affect the results, possible robot damage aside

SysId on CTRE

CTRE users: You must start and stop the CTRE signal logger for the test to provide accurate data, and the hoot logs that are created must be converted to WPILOG files in Phoenix for the SysId tool to be able to view them (more on that later).

The YAMS SysId command can be altered to automatically start and stop the signal logger:

```
public Command sysId() {  
    return m_elevator  
        .sysId(  
            Volts.of(magnitude:12),  
            Volts.of(magnitude:12).per(Second),  
            Second.of(magnitude:30))  
        .beforeStarting(  
            () -> SignalLogger.start()  
        ).finallyDo(  
            () -> SignalLogger.stop());  
}
```

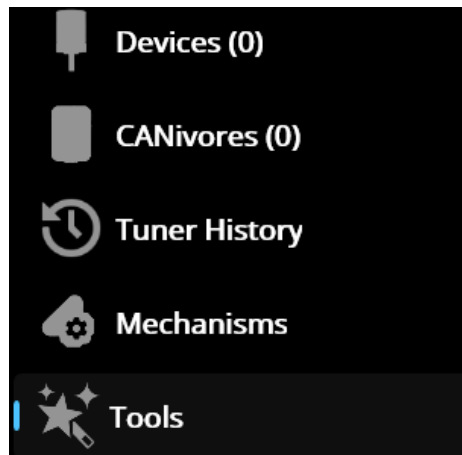

Running SysId

Bind the SysId command to a `.whileTrue()` button so you can easily stop the routine by releasing the button in case one of your configs causes the test to go wrong. However, IF YOU LET GO OF THE BUTTON YOU MUST DISREGARD THAT LOG AND RESTART THE TEST. Stopping the command before all 4 SysId routines run will result in invalid data

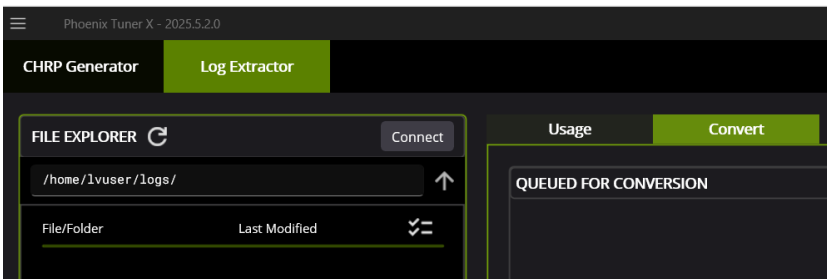
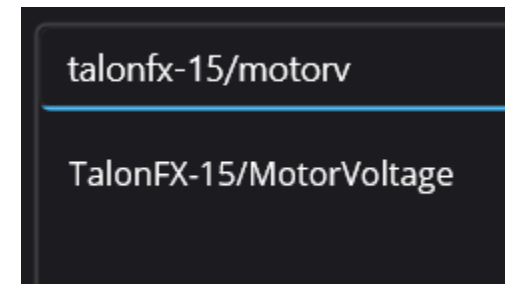
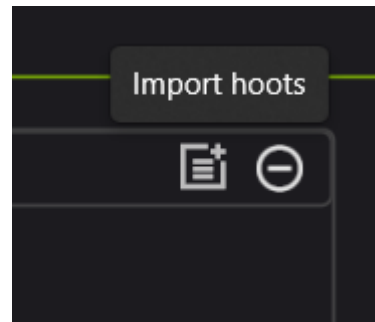
The log file will be saved in the "logs" folder of your robot project. For Thrifty Nova users, the log file will be a WPILOG that can be opened in SysID. Running SysId on CTRE hardware generates a hoot log, and REV generates a REVlog, both of which must be converted

Converting Hoot Logs

In Phoenix, open "Tools" then select "Log Extractor" and "Convert"

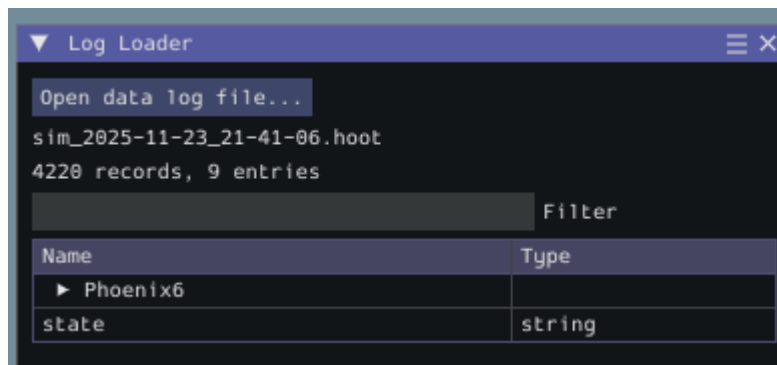


Next, import the hoot log, select the file, and click "Scan for signals." The signals you are State, Motor Voltage, Velocity, and Position. Make sure to only select the signals for the motor your mechanism uses (if a follower is configured, use the leader). The state is not stored under any motor. Once you have those 4 signals selected, click "convert" in the bottom right corner.

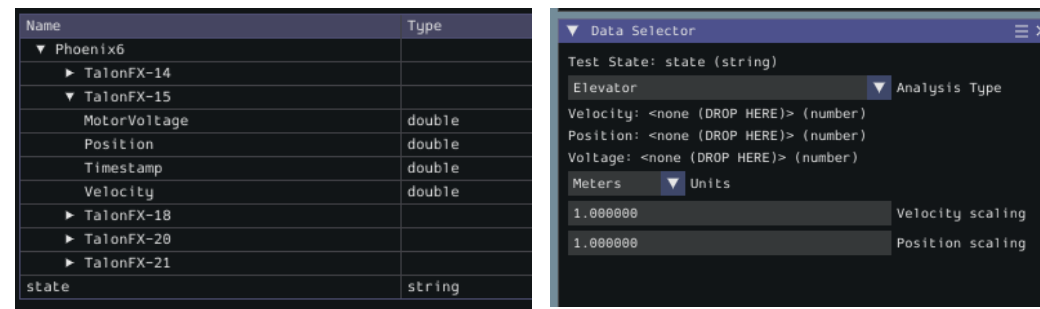


Opening The SysId Log

Open the SysId program (part of your WPILib installation) and open the log file. It should look something like this once you do so:



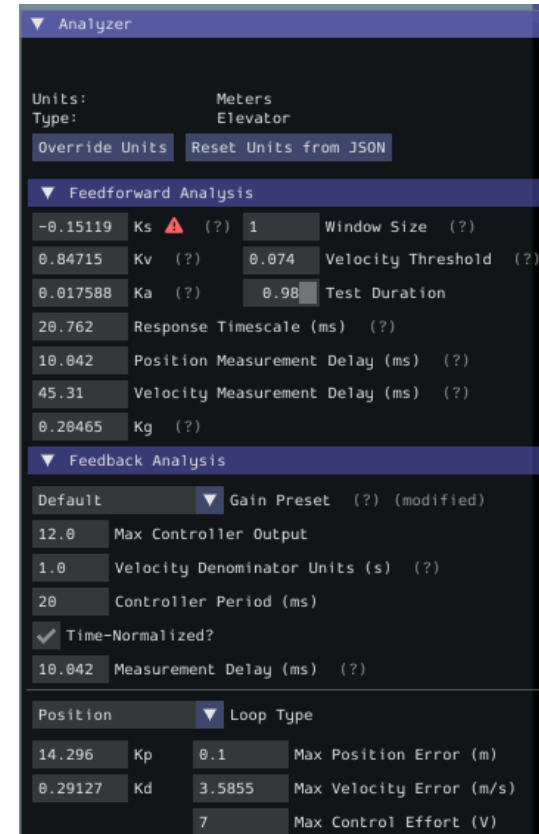
Drag the "state" field into the "Test State" data selector, and then you will see more options pop up. Drag the MotorVoltage, Position, and Velocity fields into the selector and hit load. In the analyzer, you will likely need to change the loop type from Velocity to Position



Interpreting the Data

Once you do those steps, the analyzer should show you your feedback and feedforward gains. YAMS takes care of unit conversions so your gain preset should always be Default

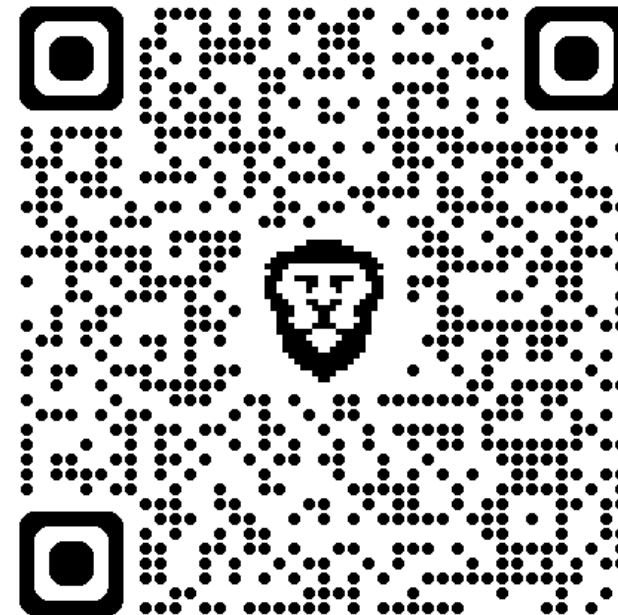
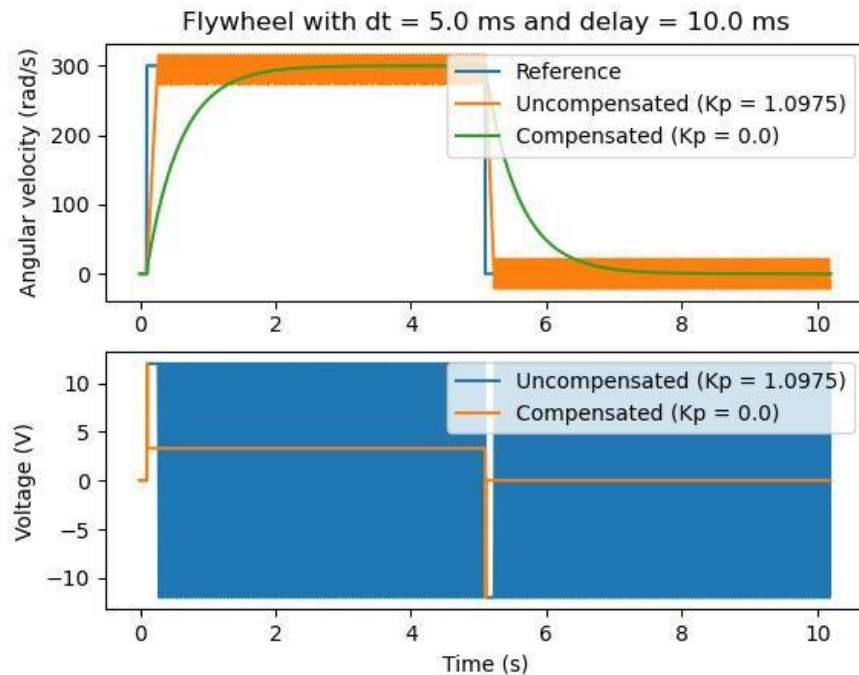
Run SysId in sim first to get a feel for what is going to happen when it runs, and to validate that your mechanism is configured correctly (if it isn't then the gains will be wrong)



Note: you should expect an invalid result for kS in sim because there is no friction in sim

THE FUTURE! LQR and State-Space Control

This is not supported in YAMS yet. In the future we will support LQR and it will be as easy to implement as everything else!



Using External Encoders

You can only use alike vendor absolute encoders in YAMS, like the `.getAbsoluteEncoder` method from Spark's and `CANCoder`'s for CTRE products.

```
SmartMotorControllerConfig smartMotorControllerConfig = new SmartMotorControllerConfig( subsystem: this)
    .withExternalEncoder(smax.getAbsoluteEncoder())
    .withExternalEncoderInverted(false)
    .withExternalEncoderGearing(new MechanismGearing( reductionRatio: 0.5)) // 1:2 -> input:output
    .withExternalEncoderZeroOffset(Degrees.of( magnitude: 0))
    .withUseExternalFeedbackEncoder( useExternalEncoder: true)
```

External Gearing

External gearing is only supported in the instances where the absolute encoder is geared UP.

AN ABSOLUTE ENCODER THAT REQUIRES MORE THAN ONE ROTATION WILL NOT READ CORRECTLY.

```
SmartMotorControllerConfig smartMotorControllerConfig = new SmartMotorControllerConfig( subsystem: this)
    .withExternalEncoder(smax.getAbsoluteEncoder())
    .withExternalEncoderInverted(false)
    .withExternalEncoderGearing(new MechanismGearing( reductionRatio: 0.5)) // 1:2 -> input:output
    .withExternalEncoderZeroOffset(Degrees.of( magnitude: 0))
    .withUseExternalFeedbackEncoder( useExternalEncoder: true)
```

What is Zero Offsets?

- The zero offset is used when a mechanism has an absolute encoder mounted on it. Absolute encoders always report a fixed angle based on their physical orientation, which usually doesn't line up perfectly with the mechanism's real zero position.

- The zero offset lets you tell the software where "zero" should actually be. You measure the encoder's reading when the mechanism is at its true home position, then pass that value into `.withExternalEncoderZeroOffset( angle/distance here )`

withExternalEncoderZeroOffset

```
public SmartMotorControllerConfig withExternalEncoderZeroOffset(Distance distance)
```

Set the zero offset of the `SmartMotorController`'s external Encoder.

Parameters:

distance - Zero offset in distance.

Returns:

`SmartMotorControllerConfig` for chaining.

withExternalEncoderZeroOffset

```
public SmartMotorControllerConfig withExternalEncoderZeroOffset(Angle angle)
```

Set the zero offset of the `SmartMotorController`'s external Encoder.

Parameters:

angle - `Angle` to 0.

Returns:

`SmartMotorControllerConfig` for chaining.

How do I Find My Zero Offset

1. Make sure you have your external encoder on your robot and in your code
2. Open an App that can read telemetry data such as Elastic and find your absolute encoder reading
3. Connect to your robot and hold your desired mechanism to the 0 position (if it is an arm, 0 degrees is parallel to the ground and 90 degrees is pointing straight upwards)
4. Record the absolute encoder output at that position and insert it into `.withExternalEncoderZeroOffset`

SmartMotorControllerConfig – Telemetry

Telemetry can be as simple or complicated as you want it

```
SmartMotorControllerConfig smartMotorControllerConfig = new SmartMotorControllerConfig(subsystem: this)
    .withTelemetry(telemetryName: "SmatMotorController Name Goes HERE", TelemetryVerbosity.HIGH)
    .withTelemetry(telemetryName: "NAME ME", new SmartMotorControllerTelemetryConfig()
        .withTelemetryVerbosity(TelemetryVerbosity.LOW)
        .withArmFeedforward()
        .withSimpleFeedforward()
        .withElevatorFeedforward()
        .withMotionProfile()
        .withSetpointPosition()
        .withMechanismPosition())
```

External Encoder is not compatible?

Periodically set or just seed the relative encoder with the absolute encoder value. The position given in the example is for the **Mechanism** NOT THE ROTOR!

```
Angle absPosition = cancoder.getAbsolutePosition().getValue();
MechanismGearing gearing = new MechanismGearing( reductionRatio: 0.5);
motor.setEncoderPosition(absPosition.times(gearing.getRotorToMechanismRatio()));
// If no gearing is required
motor.setEncoderPosition(absPosition);
```

Mixing Closed and Open Loop control

Some motor controllers do not have sufficient on-board features that provide the minimum viable control scheme for most mechanisms, like feedforwards.

For these motor controllers YAMS will run the feedback (PID), and feedforward on the robot controller (roboRIO, pre-2027; SystemCore post-2027).

The robot controller uses the *setVoltage* function every 20ms (configurable in *SmartMotorControllerConfig*) to control the motor.

This prevents the user from using *setVoltage* out-of-band on these motor controllers.

To solve this, you must stop the closed loop controller, use *setVoltage* or *setDutyCycle*, then start it back up when you're done.

Pseudo Code for Timed Based →

```
motor.setPosition(Degrees.of( magnitude: 0));  
motor.stopClosedLoopController();  
motor.setVoltage(Volts.of( magnitude: 4));  
Thread.sleep( millis: 1000);  
motor.startClosedLoopController();
```

Pseudo Code for Command Based →

```
// Set the motor position to 0deg  
run()->motor.setPosition(Degrees.zero()));  
  
// Move the motor with 4 volts for 1 second  
startRun()->{motor.stopClosedLoopController();},  
            ()->{motor.setVoltage(Volts.of( magnitude: 4));} Command  
            .withTimeout(Seconds.of( magnitude: 1)) ParallelRaceGroup  
            .finallyDo()->{motor.startClosedLoopController();});  
// Stop the closed loop controller at the start, then start it at the end  
// Motor will go back to previous setpoint (0deg) after command ends.
```

Mixing Closed and Open Loop control

Simulation and Telemetry Iteration

To ensure simulation and telemetry are updated you MUST include *.updateTelemetry()* in your Subsystem's periodic method AND *.simIterate* in your Subsystem's simulationPeriodic method!

```
@Override
public void periodic() { motor.updateTelemetry(); }

@Override
public void simulationPeriodic() { motor.simIterate(); }
```

Simulation and Telemetry Iteration

To ensure simulation and telemetry are updated you MUST include *.updateTelemetry()* in your Subsystem's periodic method AND *.simIterate* in your Subsystem's simulationPeriodic method!

```
@Override
public void periodic() { motor.updateTelemetry(); }

@Override
public void simulationPeriodic() { motor.simIterate(); }
```